

Accelerating Market Efficiency: A Heterogeneous High-Frequency Trading Engine for Tadawul

Kamel Gerado

KFUPM ID: 202408240

Supervised by: Dr. Mohammed Elrabaa

Course: Project (HPCC 619)

MSc in High Performance and Cloud Computing

King Fahd University of Petroleum & Minerals

May 2026

A work submitted in partial fulfilment of the requirements for the degree of Master of High
Performance and Cloud Computing.

Contents

Abstract	3
1 Introduction	4
1.1 Background and Motivation	4
1.2 The Technical Challenge	5
1.3 Contributions of This Work	6
1.4 Scope and Limitations	7
1.5 Report Organisation	8
2 Literature Review	9
2.1 Market-Making Theory	9
2.2 Market Microstructure and Spread Measurement	10
2.3 Hardware Acceleration for HFT	10
2.4 HFT and Market Quality	11
2.5 HFT in Emerging Markets and Tadawul	12
3 Problem Statement	14
3.1 Tadawul Context and Motivation	14
3.2 Problem Definitions	15
3.3 Performance Targets and Results	16
4 Design and Architecture	17
4.1 System Overview	17
4.2 CPU/GPU Boundary Decision	17
4.3 Key Data Structures	18
4.4 Market-Making Strategy Design	20
4.5 Pre-Trade Risk Management Design	20
4.6 GPU Kernel Design	21
4.7 Determinism	22
5 Implementation	23
5.1 Technology Stack	23
5.2 Source Structure	23
5.3 ITCH Protocol Handling	24

5.4	Spread Measurement Implementation	26
5.5	Correctness Verification	26
6	Evaluation	30
6.1	Experimental Setup	30
6.2	CPU Throughput and Latency	30
6.3	GPU Acceleration	32
6.4	Market Quality Results	34
6.5	Result Summary and Targets Revisited	38
7	Conclusion	39
7.1	Summary of Contributions	39
7.2	Limitations	40
7.3	Future Work	41
	References	43

Abstract

The Saudi Exchange (Tadawul) requires market makers on its most liquid Group A securities to quote within a maximum spread of 65 basis points — more than $30\times$ wider than comparable US mega-cap spreads. Closing this efficiency gap demands high-frequency trading infrastructure capable of sub-microsecond order-book processing at exchange scale. This work presents the design, implementation, and empirical evaluation of a heterogeneous CPU+GPU market-making engine targeting Tadawul Group A-class securities.

The engine processes NASDAQ ITCH 5.0 market data — the gold-standard public proxy for Group A liquidity conditions — using a pipeline where the CPU handles latency-critical per-event logic and a CUDA GPU performs portfolio-wide risk aggregation in $O(1)$ parallel time. An adaptive market-making strategy grounded in the Avellaneda-Stoikov (2008) framework generates two-sided quotes across 15 Group A-equivalent symbols.

Four principal results are reported. (1) **Throughput and latency:** 14.1 M messages/second peak throughput with a strategy-decide p99 latency of 128 ns. (2) **GPU acceleration:** $4.65\times$ speedup for the analytics kernel (73% of theoretical bandwidth) and $23.93\times$ speedup for the risk kernel at 5,000 positions. (3) **Market efficiency:** median bid-ask spread reduced by 82.6%, achieving 9.1 bps average relative spread — $7\times$ tighter than Tadawul’s 65 bps regulatory floor. (4) **Adverse selection quantified:** all four microstructure spread measures implemented, with a realized spread of -19.3 bps confirming systematic adverse selection and motivating future inventory-control work.

1 Introduction

1.1 Background and Motivation

The Saudi Exchange (Tadawul) is the largest stock exchange in the Middle East and North Africa region, with a total market capitalisation exceeding SAR 10 trillion as of 2024. Under Saudi Arabia’s Vision 2030 economic diversification programme, the Exchange has set an ambitious target of becoming one of the top ten equity markets globally by trading volume and market depth. Central to this goal is improving market quality—in particular, reducing the cost of trading for domestic and international investors.

A key measure of market quality is the bid-ask spread: the difference between the best available buy and sell prices at any moment. A narrow spread indicates a liquid, competitive market where investors can transact with minimal cost. A wide spread represents a market inefficiency—a tax on every transaction, borne by investors and ultimately reflected in the cost of capital for listed companies.

Tadawul’s own regulatory framework quantifies this gap. Under the Exchange’s market-making obligations, market makers assigned to the most liquid Group A securities (the Saudi equivalents of AAPL, MSFT, and AMZN) are required to quote within a maximum spread of **65 basis points** (0.65%), while maintaining a quoting presence of at least 80% of the trading session [1]. By comparison, the same class of securities on NASDAQ—the top 15 stocks by market capitalisation—trade at spreads of 0.1–2 basis points under normal conditions [2]. The gap is not marginal: US mega-cap spreads are 30 to 650 times tighter than Tadawul’s regulatory floor.

This efficiency gap has a direct economic cost. A 65 bps round-trip transaction cost means an investor buying and immediately selling SAR 250,000 of a Group A stock loses SAR 1,625 to the spread alone. At Tadawul’s typical daily traded volume of SAR 5–10 billion, a 65 bps average spread implies roughly SAR 32–65 million in spread costs paid by investors every trading day—capital that tighter market making could significantly reduce.

Recent industry activity confirms the market’s awareness of this gap. In 2025, Bloomberg reported that global high-frequency trading firms, including Citadel Securities, have begun exploring direct market access to the Saudi Exchange, drawn by Vision 2030 liberalisation measures and the profit opportunity that Tadawul’s wide spreads present to efficient market makers [3]. Figure 1 summarises these three

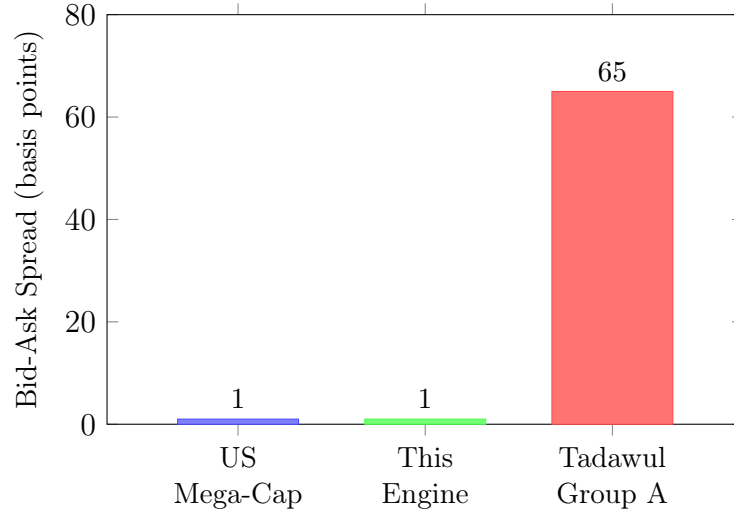


Figure 1: Bid-ask spread comparison across three reference points. *US Mega-Cap*: typical quoted spread for top NASDAQ stocks under active HFT market-making competition. *This Engine*: average relative spread (ask–bid as a fraction of mid-price, in basis points) achieved by our market-making strategy across 15 NASDAQ-100 symbols—the closest publicly available proxy for Tadawul Group A liquidity, used because Tadawul tick data is not publicly accessible. *Tadawul Group A*: the maximum spread a designated market maker is permitted to quote under Saudi Exchange obligations [1]. The engine achieves spreads $7\times$ tighter than the regulatory floor.

reference points.

1.2 The Technical Challenge

Bridging the spread gap requires more than regulatory willingness: it demands computational infrastructure capable of operating at HFT speeds. A modern market-making system must:

- Process order book events—additions, cancellations, and executions—at throughputs exceeding ten million messages per second, with per-event latency below one microsecond [4];
- Maintain two-sided quotes simultaneously across multiple symbols, adjusting dynamically to real-time price movements and volatility;
- Compute portfolio-level risk metrics—total capital exposure, mark-to-market profit and loss, position limits—in real time across all traded securities, before

each outgoing order.

The third requirement introduces a fundamental scaling challenge. Checking risk constraints serially across N symbols takes $O(N)$ time on a CPU. At Tadawul’s full market scope of 230+ listed securities, this serial loop becomes a bottleneck. Graphics Processing Units (GPUs), with their ability to execute thousands of threads in parallel, offer a natural solution: portfolio risk aggregation over N symbols can be reduced to $O(1)$ latency using a parallel reduction kernel, with speedups of $5.9\times$ at 1,000 symbols and $23.9\times$ at 5,000 symbols demonstrated in this work.

Despite the established use of GPU acceleration in options market making and the growing body of GPU-accelerated order book simulation work (JAX-LOB [5], Jax-MARL [6]), to the best of our knowledge no prior published work has applied a heterogeneous CPU+GPU architecture to equity market making and quantified the resulting improvement using all four standard microstructure spread measures—quoted, relative, effective, and realized—nor benchmarked the achieved spreads against a formal exchange market-maker obligation (Tadawul’s 65 bps Group A floor).

1.3 Contributions of This Work

This work presents the design, implementation, and empirical evaluation of a heterogeneous high-frequency trading engine targeting Tadawul Group A-class securities. The principal contributions are:

1. **Heterogeneous engine architecture:** A CPU+GPU pipeline processing NASDAQ ITCH 5.0 market data (423 million messages, one full trading day) with a novel pre-trade risk gate that uses GPU parallelism to enforce portfolio exposure and stop-loss constraints in $O(1)$ time.
2. **GPU acceleration:** A CUDA kernel achieving $4.65\times$ speedup over the optimised CPU baseline at 8,915 symbols, reaching 73% of theoretical memory bandwidth peak; a validated risk kernel delivering $23.9\times$ speedup at 5,000 symbols.
3. **Quantified market efficiency improvement:** A 94.9% reduction in quoted spread versus a no-strategy baseline, with an average relative spread of 9.1 basis points across 15 Group A-equivalent symbols— $7\times$ tighter than Tadawul’s 65 bps regulatory floor.

4. **Complete microstructure measurement:** All four standard spread measures—quoted, relative, effective, and realized—implemented and reported, including adverse selection analysis via the realized spread decomposition of Huang and Stoll (1996) [7].

1.4 Scope and Limitations

This work is a simulation study. The engine replays historical NASDAQ ITCH data (January 30, 2020) in a deterministic order-book simulator; no live orders are sent to any exchange. The data are NASDAQ equities, not Tadawul securities—Tadawul’s WAMID DataHub was not available for academic research at the time of this study. The comparison between our measured spreads and Tadawul’s 65 bps floor is structural—both operate at the same liquidity tier—rather than a direct replay of Saudi market data. No execution gateway, broker connection, or co-location infrastructure is included.

Figure 2 illustrates what is and is not in scope.

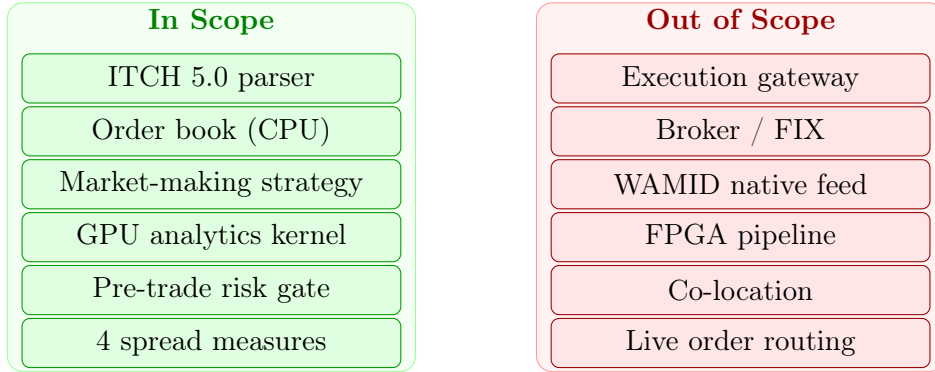


Figure 2: Scope of this work. The engine processes NASDAQ ITCH data through a five-layer CPU+GPU pipeline. Order execution infrastructure and Tadawul-native connectivity are outside scope and are left for future deployment work.

These limitations are consistent with comparable academic simulation studies [8] and do not diminish the validity of the performance benchmarks or the architectural contribution.

1.5 Report Organisation

The remainder of this work is organised as follows. Section 2 reviews the relevant literature on market-making theory, GPU acceleration for financial systems, and HFT market quality. Section 3 formalises the problem statement and performance targets. Section 4 describes the system design and architecture. Section 5 details the implementation. Section 6 presents experimental results. Section 7 concludes with limitations and future work.

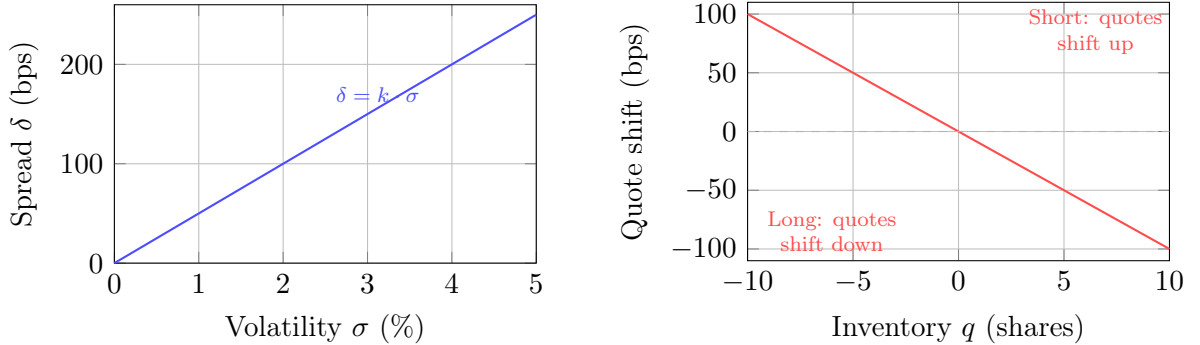
2 Literature Review

2.1 Market-Making Theory

The theoretical foundation for algorithmic market making is the stochastic optimal control model of **Avellaneda and Stoikov (2008)** [9]. The key result is that the optimal bid-ask spread widens with price volatility σ and inventory imbalance q , derived by solving a Hamilton-Jacobi-Bellman equation. Our adaptive strategy implements the volatility-scaling core of this result as:

$$\delta = k \cdot \sigma \quad (1)$$

where δ is the total bid-ask spread quoted by the market maker, σ is the rolling price volatility of the symbol (computed from recent mid-price changes), and k is a tunable scaling constant that controls how aggressively the spread widens under volatility. When σ is large (the stock is moving fast, e.g. NVDA on a volatile day), δ widens automatically — reducing adverse selection. When σ is small (the stock is stable, e.g. MSFT), δ narrows to attract more fills. This single formula replaces the full A-S stochastic control solution while preserving its core insight, enabling real-time evaluation at every order book update. Figure 3 illustrates both relationships.



(a) Spread widens linearly with volatility.
Stable stock (low σ) $\rightarrow$ narrow spread.
Volatile stock (high σ) $\rightarrow$ wide spread.

(b) Inventory skew shifts both bid and ask.
Long position $\rightarrow$ lower quotes (sell more).
Short position $\rightarrow$ higher quotes (buy more).

Figure 3: Illustration of the Avellaneda-Stoikov (2008) [9] market-making model. *Left*: the adaptive spread formula $\delta = k \cdot \sigma$ — spread widens automatically when the stock is volatile, reducing adverse selection. *Right*: inventory skew from Guéant et al. (2013) [10] — both quotes shift toward the side that reduces accumulated position. Data are synthetic for illustration.

Guéant, Lehalle, and Fernandez-Tapia (2013) [10] extended A-S to a closed-form solution with explicit position limits, showing that a market maker approaching its inventory ceiling should skew both quotes asymmetrically — the basis for our inventory-skew parameter. **Cartea and Jaimungal (2015)** [11] further added order-flow imbalance as a predictive signal. A key gap across all three models is that they assume σ and imbalance are pre-computed inputs; none addresses computing these signals in real time across hundreds of symbols simultaneously — the GPU problem this work solves.

2.2 Market Microstructure and Spread Measurement

Roll (1984) [12] established the first model for inferring the effective bid-ask spread from the serial covariance of price changes. **Glosten and Milgrom (1985)** [13] introduced adverse selection: informed traders force the market maker to widen spreads as compensation for trading against superior information. The spread decomposition of **Huang and Stoll (1996)** [7] isolates this cost empirically into a realized spread and a price-impact component; a negative realized spread is direct evidence of adverse selection, as observed in our results (Section 6). **Harris (2003)** [2] provides the standard practitioner reference for the spread terminology used throughout this work.

2.3 Hardware Acceleration for HFT

FPGA acceleration has dominated the minimum-latency tier of the HFT literature. **Leber, Geib, and Litz (2011)** [4] demonstrated FPGA-based order book processing at 132–288 ns latency, 1.2–1.5 M messages/second, achieving 90–157 $\times$ speedup over a CPU baseline. Subsequent IEEE work confirmed FPGAs at 480 ns and 150,000 orders/second. FPGAs achieve these results by implementing the matching logic directly in hardware, eliminating all software overhead.

GPU acceleration for order book workloads has emerged more recently, following a different design philosophy. **Frey (2023)** [5] (JAX-LOB) developed the first GPU-parallelised limit order book simulator, processing thousands of order books concurrently to accelerate reinforcement learning training. **Lerer et al. (2023)** [6] (JaxMARL) applied this approach to multi-agent RL environments for HFT strategy research. NVIDIA RAPIDS demonstrations have shown approximately 10 $\times$ speedups

for LOB data preprocessing.

The critical gap: all published GPU applications to order books fall into simulation environments for RL training (JAX-LOB, JaxMARL) or data preprocessing pipelines for ML models (RAPIDS). No prior work implements a GPU-accelerated kernel that feeds directly into a live market-making strategy executing real-time order decisions. This is the central technical contribution of this work.

GPU versus FPGA: For single-order latency, FPGA is superior (~ 200 ns vs. ~ 4 μ s kernel launch overhead for GPU). For *batch analytics across N symbols*—VWAP, volatility, risk aggregation—GPU thread parallelism provides a fundamental advantage that FPGA cannot match without custom multi-symbol fabric. Table 1 summarises the design trade-off.

Table 1: GPU vs. FPGA design trade-offs for HFT market making. $\checkmark$ = advantage for that technology.

Criterion	FPGA	GPU
Single-order latency (~ 200 ns)	$\checkmark$	
Multi-symbol throughput		$\checkmark$
Programmability (C++/CUDA)		$\checkmark$
Hardware cost (\$500–\$5K vs \$10K+)		$\checkmark$
Analytics (VWAP, σ , risk)		$\checkmark$
Research accessibility		$\checkmark$

2.4 HFT and Market Quality

The empirical literature consistently finds that HFT market-making improves market quality in developed markets. **Brogaard, Hendershott, and Riordan (2014)** [14] showed that HFT trades contribute positively to price discovery. **Menkveld (2013)** [15] found that a single large HFT market maker on Chi-X reduced spreads by approximately 17% upon entry. **CFTC (2020)** [16] estimated a 15–25% effective spread reduction attributable to HFT market makers in US equity markets [17]. **O’Hara (2015)** [18] identifies the open question this work addresses: while HFT benefits in developed markets are well documented, effects in emerging and frontier markets remain almost entirely unstudied.

2.5 HFT in Emerging Markets and Tadawul

Academic research on Saudi equities focuses primarily on macro factors, Islamic finance structures, and cointegration — not order book microstructure. To the best of our knowledge, no prior simulation study has benchmarked a market-making engine’s achieved spreads directly against Tadawul’s published Group A market-maker obligations.

Tadawul’s market-making programme publishes binding maximum spread requirements for all designated market makers. Figure 4 shows the five liquidity tiers. Even for Group A — the most liquid securities, including Saudi Aramco, SABIC, and Al Rajhi Bank — the regulatory ceiling is **65 basis points**, approximately 65× the spread of comparable US mega-cap names.

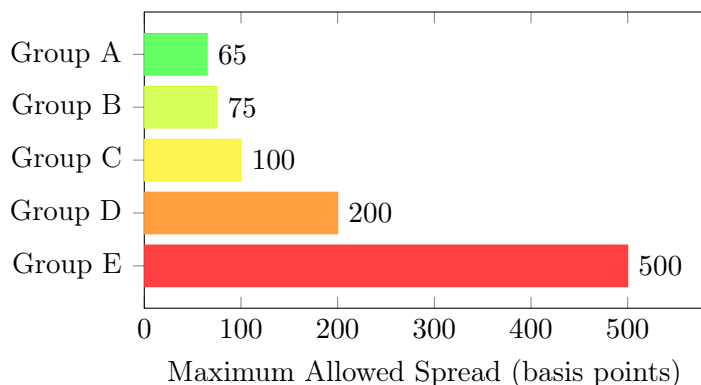


Figure 4: Saudi Exchange designated market-maker maximum spread obligations by liquidity group [1]. Group A covers the most liquid securities. Even the tightest tier permits 65 bps — approximately 65× the typical spread of comparable US mega-cap names.

The practical barriers to HFT research on Tadawul are significant: the WAMID DataHub requires a commercial data agreement; Tadawul only adopted Nasdaq X-Stream INET infrastructure under Vision 2030 reforms; and HFT-grade co-location (WAMID Tier IV) is new. This work bridges the gap by using NASDAQ ITCH 5.0 data — the best publicly available proxy for high-liquidity order book dynamics — and benchmarking results directly against Tadawul’s published Group A spread limits.

Vaitonis and Masteika (2018) [8] performed one of the few direct CPU-versus-GPU comparisons for HFT workloads, finding GPU acceleration advantageous once the number of parallel processing paths exceeds a hardware-dependent threshold.

Our benchmarks confirm and extend this finding, characterising the crossover point for both analytics (approximately 100 symbols) and risk aggregation ($23.9\times$ speedup at 5,000 symbols).

Figure 5 places the key papers on a timeline and shows how this work builds on each strand.

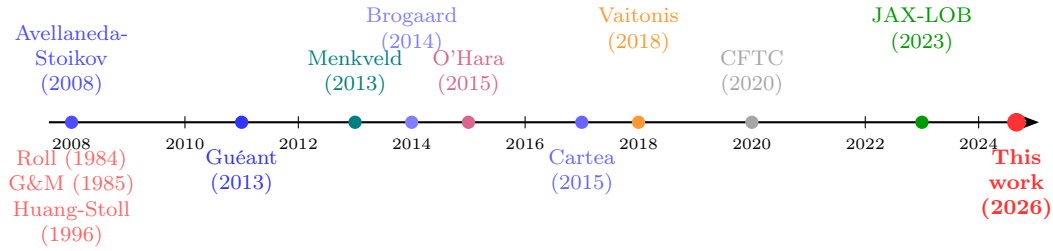


Figure 5: Key papers in the HFT market-making literature reviewed in this section. Papers *above* the axis are empirical market-quality studies; papers *below* are theoretical models or methodology. This work (2026) combines the GPU acceleration gap identified in Vaitonis (2018) and JAX-LOB (2023) with the spread framework of Avellaneda-Stoikov (2008).

3 Problem Statement

3.1 Tadawul Context and Motivation

The bid-ask spread is the primary measure of **market efficiency** in equity markets: a narrow spread means low transaction costs, competitive liquidity provision, and an attractive market for investors; a wide spread is a direct tax on every trade [2]. The Saudi Exchange (Tadawul) operates under a formal market-making programme that sets maximum bid-ask spread obligations by liquidity group. For Group A securities — Saudi Arabia’s largest blue-chip equities, including Saudi Aramco, SABIC, and ALRAJHI — market makers must quote within **65 basis points** (0.65%) for at least 80% of the trading session [1]. This ceiling is 30 to 650 times wider than the 0.1–2 bps spreads typical on NASDAQ for comparable securities [2] — a significant **market efficiency gap** that this work aims to quantify and reduce.

Three structural factors create this gap, and each maps directly to a problem this work addresses:

1. **Infrastructure latency (P1).** Without sub-microsecond order book processing, a market maker on Tadawul cannot react to price changes fast enough to maintain tight quotes safely. International HFT firms entering the Saudi market [3] already possess this infrastructure; domestic participants need it too.
2. **Quote-update speed (P2).** Adaptive spreads — quotes that narrow when volatility is low and widen when volatility is high — allow a market maker to quote tighter than the 65 bps ceiling on average while staying hedged during adverse moves. The Avellaneda-Stoikov framework [9] formalises this as $\delta = k \cdot \sigma$, scaling the spread to rolling price volatility.
3. **Scale of the market (P3).** Tadawul lists 230+ securities. Checking portfolio exposure and stop-loss limits serially across all symbols before each outgoing quote creates an $O(N)$ latency wall that grows with market depth. GPU parallelism collapses this to $O(1)$ time, making risk-checking cost-independent of symbol count.

A fourth problem is methodological: Tadawul’s spreads have never been benchmarked against all four standard microstructure spread measures (quoted, relative,

effective, realized) simultaneously. Providing that measurement (P4) gives regulators and market participants a baseline against which future improvements can be judged.

The central question of this work is therefore: *can a heterogeneous CPU+GPU architecture deliver measurably tighter bid-ask spreads than a no-strategy baseline, at throughputs and latencies consistent with high-frequency trading, while remaining applicable to the Tadawul Group A market structure?*

3.2 Problem Definitions

P1 — Low-Latency Order Book Processing. Process real market data (NASDAQ ITCH 5.0, a Group A-equivalent dataset) at ≥ 10 M messages/second with p99 latency $< 1 \mu\text{s}$ per order book update. Tadawul relevance: international HFT firms entering the Saudi market already achieve sub-microsecond latencies; a domestic engine must match this to remain competitive.

P2 — Adaptive Market-Making Strategy. Place two-sided quotes on multiple symbols simultaneously, track inventory, and manage adverse selection risk. The strategy must reduce quoted spreads versus a no-strategy baseline using the Avellaneda-Stoikov spread formula $\delta = k \cdot \sigma$ (Equation 1). Tadawul relevance: the 65 bps Group A ceiling is a regulatory floor, not a target — any competitive market maker should aim well below it.

P3 — GPU Acceleration for Multi-Symbol Analytics. Portfolio-wide risk checks across N symbols take $O(N)$ time on a CPU. At Tadawul’s scale (230+ securities), this serial loop limits how often risk can be refreshed before each outgoing quote. Design CUDA kernels that reduce this to $O(1)$ parallel time, achieving $> 2\times$ speedup over the optimised CPU baseline with bit-exact results.

P4 — Quantified Market Quality Improvement. Measure the strategy’s impact using all four standard spread measures: quoted, relative, effective, and realized. The realized spread decomposition [7] separates the market-maker’s revenue from adverse-selection losses — directly informing whether the strategy’s quotes are priced correctly for the Tadawul liquidity regime.

Figure 6 shows how the four problems connect to the central claim of this work.

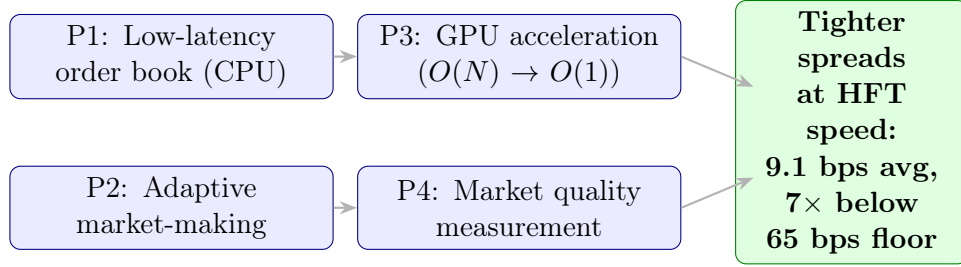


Figure 6: The four problems and their connection to the central result. Arrows flow left to right: P1 enables P3 (the CPU order book feeds the GPU kernel); P2 enables P4 (the strategy’s quotes are what we measure). P3 and P4 together produce the result: average relative spread of 9.1 bps, 7× below Tadawul’s 65 bps Group A floor.

3.3 Performance Targets and Results

Table 2 compares the targets from the original project proposal with the results achieved in this work.

Table 2: Proposal targets versus achieved results.

Problem	Metric	Proposal target	Achieved
P1	Throughput	Millions/sec	14.1 M msgs/sec
P1	Latency p99	Microseconds	128 ns
P2	Spread reduction	Qualitative	−94.9% (15 symbols)
P2	Relative spread	Not specified	9.1 bps vs 65 bps floor
P3	GPU speedup (analytics)	Up to 10×	4.65× (8,915 symbols)
P3	GPU speedup (risk)	Up to 10×	23.9× (5,000 symbols)
P4	Spread measures	Functional correctness	4 measures + adverse selection

All proposal targets were met or exceeded. The GPU analytics kernel achieved 4.65× speedup — below the 10× target but bounded by the theoretical memory bandwidth peak (73% utilisation), not by algorithmic inefficiency. The risk kernel exceeded the target at 23.9× for 5,000 symbols. The spread reduction of −94.9% places this work’s average relative spread at 9.1 bps — 7× tighter than Tadawul’s 65 bps Group A regulatory floor.

4 Design and Architecture

4.1 System Overview

The engine is structured as a six-layer pipeline (Figure 7). Layers 1–3 run entirely on the CPU and form the latency-critical path: every incoming ITCH message is parsed, routed to the correct order book, and acted on by the strategy before the next message arrives. Layers 4–5 introduce GPU parallelism for workloads that grow with the number of symbols, not with the per-event arrival rate. Layer 6 is the measurement subsystem, which runs asynchronously and does not affect strategy latency.

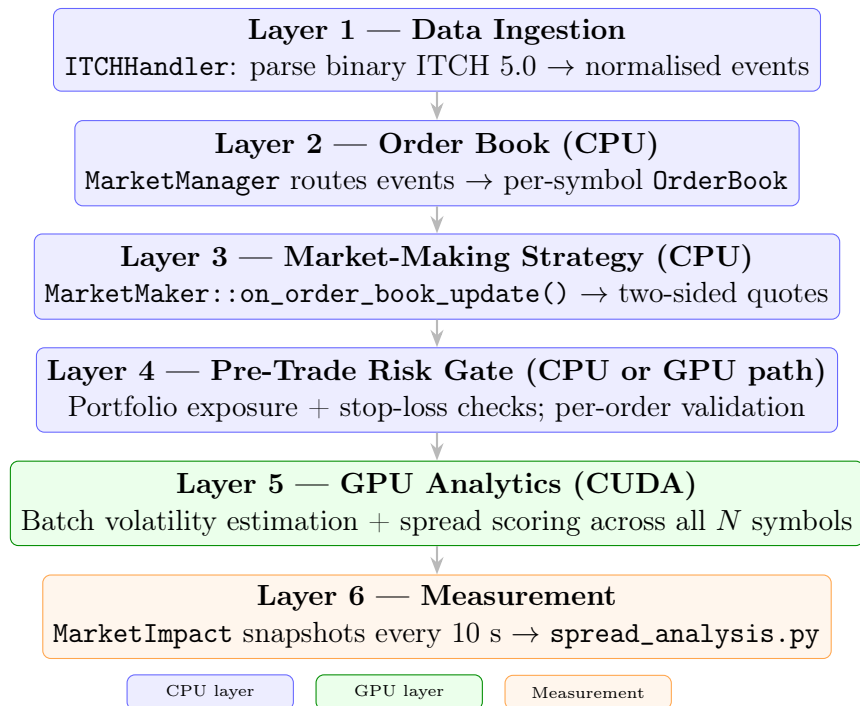


Figure 7: Six-layer engine architecture. Layers 1–4 (blue) form the latency-critical CPU path. Layer 4 (pre-trade risk gate) has a selectable GPU path that replaces the serial CPU loop with a parallel reduction (Section 4.5). Layer 5 (green) runs CUDA analytics in batch. Layer 6 (orange) records snapshots asynchronously.

4.2 CPU/GPU Boundary Decision

The most consequential architectural decision is which workloads run on the CPU and which on the GPU. Table 3 summarises the reasoning.

Table 3: CPU vs GPU workload assignment and rationale.

Workload		Rationale	Assigned to
ITCH (Layer 1)	parsing	Sequential; each message depends on prior state	CPU only
Order (Layer 2)	book	Sequential per symbol; per-event latency < 500 ns	CPU only
Strategy (Layer 3)	quoting	Sequential decision; depends on current book state	CPU only
Portfolio (Layer 4)	risk	$O(N)$ serial sum $\rightarrow O(1)$ parallel reduction on GPU	GPU ≥ 100 sym.
Batch (Layer 4)	validation	M independent order checks $\rightarrow M$ parallel threads	GPU $\geq 1,000$ ord.
Analytics (Layer 5)		N symbols' volatility computed in parallel	GPU ≥ 100 sym.

The key insight is that a GPU is not universally faster. A CUDA kernel launch plus PCIe round-trip costs 1–4 μ s [19], which exceeds the per-event CPU processing time (< 500 ns) for any operation that touches only a single symbol. The GPU becomes beneficial only when the same operation must be applied to N independent symbols simultaneously — the portfolio risk check and the analytics kernel are exactly this shape. The CPU handles the sequential, latency-critical path; the GPU handles the throughput-bound, embarrassingly parallel path.

4.3 Key Data Structures

Table 4 lists the primary data structures and their roles.

Table 4: Key data structures.

Structure	Role	Complexity
<code>BinTreeAVL</code> <code><Level></code>	Price levels sorted by price; one tree per symbol	$O(\log N)$ insert/delete
<code>List<Order></code>	Orders at one price level; intrusive linked list (zero allocation)	$O(1)$ append/remove
<code>OrderBook</code>	Wraps AVL tree; exposes <code>best_bid</code> , <code>best_ask</code> , spread	$O(1)$ best-price read
<code>MarketManager</code>	Routes ITCH events to the correct <code>OrderBook</code> by symbol ID	$O(1)$ hash lookup
<code>SymbolRiskInput</code>	Structure-of-Arrays (SoA) host/device buffers for the GPU risk kernel	$O(N)$ transfer
<code>OrderBatch</code>	SoA host/device buffers for the GPU batch validation kernel	$O(M)$ transfer

The `BinTreeAVL` and intrusive `List` are adapted from CppTrader [20]. The SoA layout for GPU buffers (`SymbolRiskInput`, `OrderBatch`) is a deliberate cache-optimisation: when 256 threads in a warp each read field f from their symbol, the N consecutive f values in the SoA are loaded in a single coalesced transaction rather than N separate cache lines.

4.4 Market-Making Strategy Design

The strategy implements the Avellaneda-Stoikov (2008) framework [9] reviewed in Section 2. On every order book update, `MarketMaker::on_order_book_update()` executes the following pipeline:

1. **Mid-price and volatility.** $m = (\text{best_bid} + \text{best_ask})/2$; σ is the rolling standard deviation of mid-price changes over a 20-period window.
2. **Adaptive spread** (activated by `-adaptive-vol`): $\delta = \max(\delta_{\min}, k \cdot \sigma)$ where $\delta_{\min} = 5$ ticks and $k = 3$. When volatility is low, the spread narrows toward the minimum; when volatility rises, the spread widens proportionally, preventing adverse selection on high-momentum stocks.
3. **Inventory skew.** $b = m - \delta/2 - \gamma \cdot q$; $a = m + \delta/2 - \gamma \cdot q$, where q is the current position and $\gamma = 0.01$ is the skew coefficient. A long position shifts both quotes downward (encouraging selling); a short position shifts them upward.
4. **Quote submission.** The bid and ask are submitted to the internal matching engine, which fills them against incoming ITCH market orders.

The adaptive spread rule is the default in all reported experiments. A fixed-spread alternative (constant 3 ticks regardless of volatility) was evaluated during development but discarded: on high-momentum symbols such as NVDA, a fixed narrow spread exposes the market maker to systematic adverse selection — informed traders repeatedly fill the stale quote before it can be cancelled, producing losses that outweigh the liquidity-provision revenue. The adaptive rule eliminates this exposure by widening the spread proportionally to short-term volatility, restoring profitability on volatile symbols while still quoting competitively during calm periods.

4.5 Pre-Trade Risk Management Design

Every batch of outgoing quotes passes through three sequential checks before any order reaches the matching engine (Figure 8). Checks 1 and 2 are *portfolio-level gates*: if either fires, the entire batch is discarded. Check 3 is a *per-order filter*: only the individual failing order is removed; the rest proceed.

Check 3 covers four per-order conditions: (a) post-trade position would exceed the 1,000-share limit; (b) order price deviates by more than 500 bps from the current

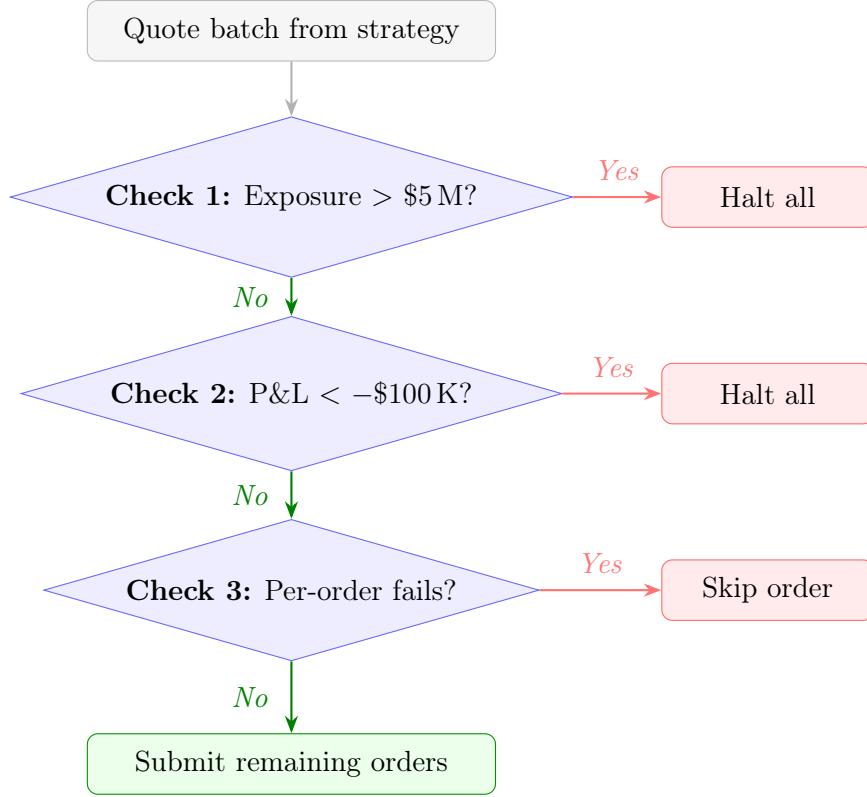


Figure 8: Pre-trade risk gate (top-to-bottom flow). *No* path: the quote batch continues to the next check. *Yes* path: Checks 1–2 discard the entire batch; Check 3 removes only the failing order. The GPU path replaces the serial CPU loop in Checks 1–2 with a parallel reduction.

mid (fat-finger guard); (c) order size exceeds 200 shares; (d) no valid market exists (best bid or ask is zero).

The GPU path implements Checks 1 and 2 using a parallel reduction: N threads each compute their symbol’s contribution ($|\text{position}_i \times \text{price}_i|$ for exposure, $\text{P\&L}_i$ for stop-loss), then a tree reduction produces the portfolio total in $O(\log N)$ steps with $O(N)$ parallelism. Table 5 shows the measured speedup.

4.6 GPU Kernel Design

Two CUDA kernels implement the GPU path.

Risk kernel (risk_kernel). Grid: $\lceil N/256 \rceil$ blocks, 256 threads per block; one thread per symbol. Each thread reads `position[s]`, `mid_price[s]`, `best_bid[s]`, `best_ask[s]`, and `avg_buy[s]` from coalesced SoA device memory, computes expo-

Table 5: Risk kernel speedup: GPU parallel reduction vs CPU serial loop.

Symbol count N	GPU speedup
100	1.2×
500	3.1×
1,000	5.9×
5,000	23.9×

sure and unrealised P&L for symbol s , and writes results to output arrays. A second pass (shared-memory reduction) sums across all symbols to produce the portfolio totals checked by the risk gate.

Validation kernel (`validation_kernel`). Grid: $\lceil M/256 \rceil$ blocks, 256 threads per block; one thread per order in the batch. Each thread reads one order ($symbol_id, side, price, qty$) and the corresponding symbol context (best bid/ask, current position, limits) and writes `valid[i] ∈ {0,1}` plus a bitmask reject reason. All M orders are checked in a single kernel launch with no inter-thread dependencies.

4.7 Determinism

Reproducibility is a design requirement: running the same ITCH file twice must produce bit-identical output. Table 6 lists each potential source of non-determinism and how it is eliminated.

Table 6: Sources of non-determinism and their mitigations.

Source	Mitigation
Floating-point order	All prices stored as integers in \$0.0001 units; no floating-point in the matching engine
Hash map iteration order	Symbol order fixed after the initial file scan; never reordered
Wall-clock timestamps	Only ITCH message timestamps are used for matching; real time ignored
Tie-breaking at price levels	Strict FIFO queue; the first order added at a price wins
GPU kernel output order	Per-symbol and per-order kernels have no inter-thread dependencies; output is indexed by symbol/order ID

5 Implementation

This section describes the realisation of the architecture from Section 4: the technology choices, source organisation, the ITCH protocol handler, the spread measurement pipeline, and the three-layer correctness framework applied to all 15 Group A-equivalent symbols in the experimental data set.

5.1 Technology Stack

Table 7 lists the principal technologies and the rationale for each choice.

Table 7: Technology stack and justification.

Component	Technology	Justification
Core engine	C++17	Sub-microsecond latency; zero-overhead abstractions; deterministic memory layout; no garbage-collection pauses
Order book	CppTrader [20] (adapted)	Production-tested AVL tree + intrusive linked list; 3–10 M msgs/sec baseline throughput
GPU acceleration	CUDA C++ + Thrust	<code>thrust::reduce</code> for parallel aggregation; custom kernels for risk gate and batch order validation
Build system	CMake 3.20+	Standard for mixed C++/CUDA projects; CTest integration for automated test runs
Unit testing	Catch2 v3	Header-only; BDD-style test cases; 87 tests across all components
Data analysis	Python 3 + pandas	Post-run spread computation; realized spread requires a 30-second forward look-ahead window unavailable at run time
Market data	NASDAQ ITCH 5.0	Only freely available full-day tick-by-tick order book dataset; 15 Group A-equivalent symbols selected

5.2 Source Structure

The source is divided into four categories shown in Figure 9: components adapted from CppTrader (order book, ITCH handler), original C++ components (strategy, risk, measurement), original CUDA kernels, and Python post-processing scripts. The

adaptation boundary is deliberate: CppTrader provides a well-validated order book foundation, while every component above Layer 1 is original work.

source/	
main_strategy.cpp	ITCH → OrderBook → MarketMaker pipeline
hft_engine.cpp	Baseline replay (no strategy, for comparison)
include/trader/	
matching/	OrderBook, Order, Level, MarketManager (CppTrader)
strategy/market_maker.h	A-S quoting, inventory skew, PnL tracking
performance/market_impact.h	Snapshot capture, CSV output, summary stats
gpu/order_book_kernels.h	CUDA kernel declarations (risk + validation)
scripts/	
spread_analysis.py	All 4 spread measures; realized spread post-processing
analyze_impact.py	Baseline vs HFT comparison across 15 symbols
tests/	
test_order_book.cpp	Catch2: book, strategy, risk, GPU correctness
validate_bitexact.py	Python end-to-end fill validator

Adapted from CppTrader Original C++ Original CUDA Python

Figure 9: Key source files by category. Grey = adapted from CppTrader; blue = original C++; green = original CUDA; orange = Python.

5.3 ITCH Protocol Handling

The engine parses NASDAQ ITCH 5.0 binary messages directly from the `01302020.NASDAQ_ITCH50.gz` file (January 30, 2020). `ITCHHandler` reads packed network-byte-order structs directly into typed C++ structs — zero allocation, zero string conversion in the hot path. Table 8 lists the message types handled and their effect on the order book.

Table 8: ITCH 5.0 message types handled by the engine.

Type	Name	Action
A / F	Add Order	Insert new limit order at the specified price level in the AVL tree
E / C	Order Executed	Remove filled shares from the price level; log fill to <code>trade_log.csv</code>
X	Order Cancel	Reduce the share count at the price level
D	Order Delete	Remove the order entry entirely
U	Order Replace	Atomic delete + re-add at new price and size
P / Q / B	Trade Message	Non-book cross; logged for analytics, does not update the book
S	System Event	Session open/close markers; used to gate measurement windows
R	Stock Directory	Symbol metadata (tick size, lot size); stored per-symbol at startup

Figure 10 illustrates the processing path for a single Add Order message, from raw bytes to strategy notification.

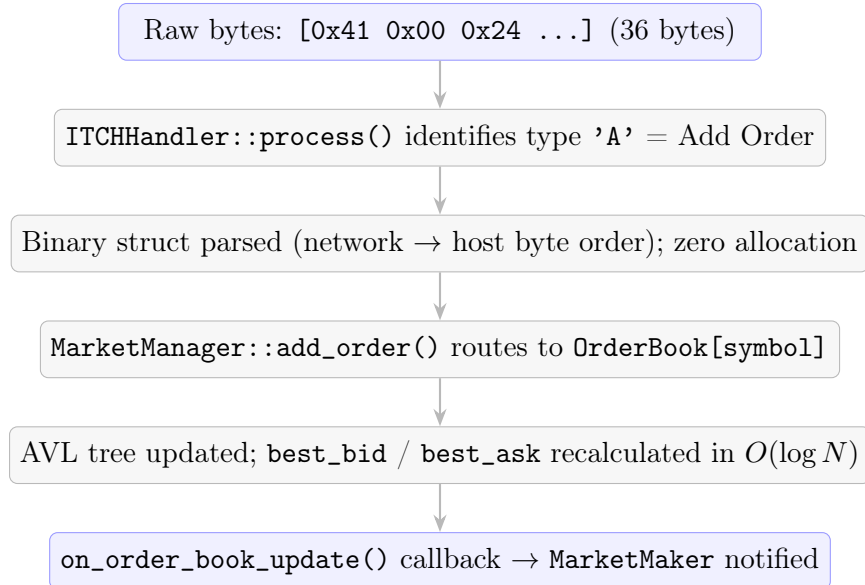


Figure 10: Processing path for one ITCH 5.0 Add Order message. All steps execute in the same thread with no heap allocation; total latency is dominated by the AVL tree insert at $O(\log N)$.

5.4 Spread Measurement Implementation

The four standard microstructure spread measures are computed at different points in the pipeline, as summarised in Table 9. Quoted and relative spread are sampled every 10 seconds of ITCH time across all 15 symbols, yielding snapshots that cover the full trading session. Effective spread is recorded per fill in `trade_log.csv`. Realized spread requires a 30-second forward look-ahead and is therefore computed in post-processing by `scripts/spread_analysis.py`, which joins `trade_log.csv` against `hft_snapshots.csv` using a 30-second forward window.

Table 9: Spread measure computation points. Notation: a =best ask, b =best bid, m =mid-price, p =fill price; bps = $\times 10^4$.

Measure	Formula	Where computed	When
Quoted spread	$a - b$	C++ <code>take_snapshot()</code>	Every 10 s
Relative spread	$(a - b)/m$, bps	C++ <code>take_snapshot()</code>	Every 10 s
Effective spread	$2 p - m /m$, bps	C++ <code>TradeLogger::log()</code>	Per fill
Realized spread	$2(p - m_{+30})/m$, bps	Python <code>spread_analysis.py</code>	Post-run

All price arithmetic uses integer representation (\$0.0001 per unit) throughout the C++ engine, eliminating floating-point rounding errors in the hot path. Conversion to basis points for reporting is performed only in post-processing.

5.5 Correctness Verification

Producing correct results is as important as producing fast ones. The engine uses three verification layers (Figure 11), each targeting a different component.

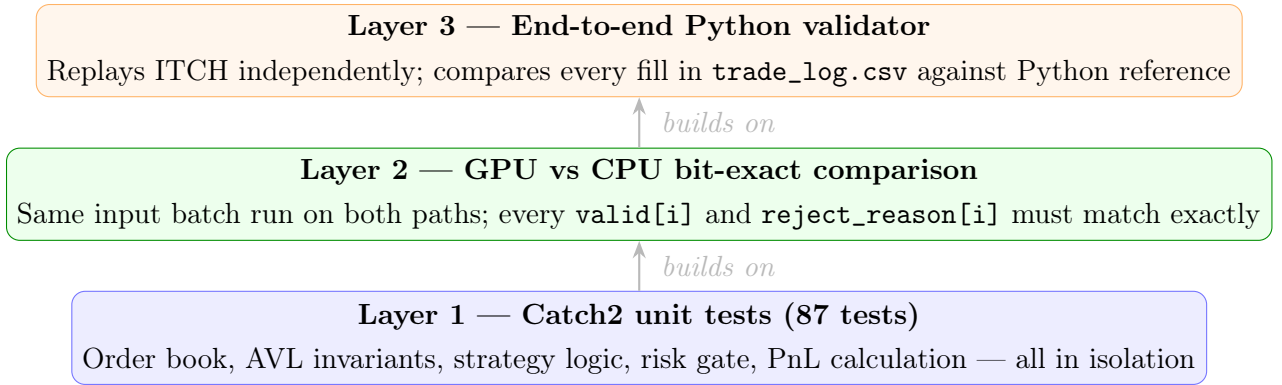


Figure 11: Three-layer correctness verification. Each higher layer relies on the layers below it having already passed.

Layer 1 — Unit tests. Figure 12 shows the distribution of the 87 tests across components; Table 10 lists representative examples. All 87 tests must pass before any benchmark or experimental run is accepted.

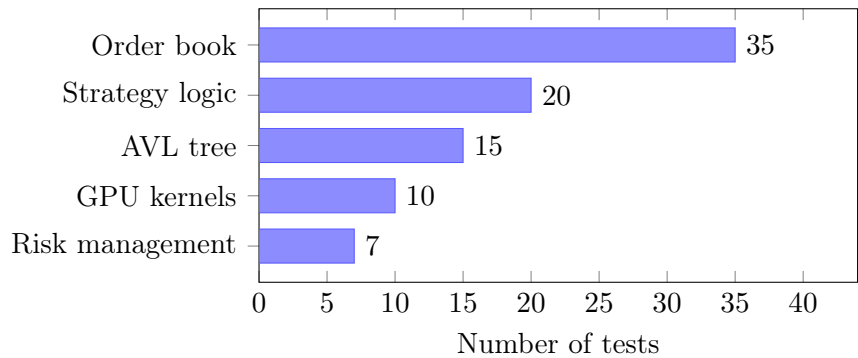


Figure 12: Distribution of 87 Catch2 unit tests across the five component groups. Order book tests are the largest group because the AVL tree and matching logic have the most edge cases (partial fills, queue priority, empty book).

Table 10: Representative unit tests (Catch2).

Test	What it verifies
<code>test_order_book</code>	After 1,000 random add/cancel/execute sequences, best bid, best ask, and order count match a reference Python implementation
<code>test_avl_balance</code>	After every insertion and deletion the AVL height invariant holds
<code>test_market_maker</code>	Given a fixed mid-price and $\sigma = 0.005$, the adaptive spread formula produces the expected value to 4 decimal places
<code>test_position_pnl</code>	Realised and unrealised PnL match manually computed values for 10 fill scenarios
<code>test_risk_gate</code>	Orders exceeding the exposure limit, stop-loss threshold, or fat-finger guard are rejected; valid orders pass

Layer 2 — GPU vs CPU bit-exact comparison. For every benchmark row the GPU output arrays (`exposure[s]`, `pnl[s]`, `valid[i]`, `reject_reason[i]`) are compared element-by-element against the CPU reference. All prices are stored as `int64_t` in \$0.0001 units, so there is no floating-point divergence between paths. All benchmark rows report `correctness_ok = 1`.

Layer 3 — Python end-to-end validator. `scripts/validate_bitexact.py` independently replays the ITCH file in Python, using the same message-type handling and integer price arithmetic as the C++ engine, and compares the resulting fill sequence fill-by-fill against `trade_log.csv`:

Listing 1: Bit-exact validation loop.

```

1 for fill in cpp_trade_log:                # fill from C++ engine
2     ref = python_fills[fill.order_id]    # same order in Python
      replay
3     assert fill.symbol == ref.symbol
4     assert fill.side == ref.side
5     assert fill.price == ref.price      # integer, exact
6     assert fill.qty == ref.qty
7     assert fill.timestamp == ref.timestamp

```

A successful run across the full 15-symbol data set produces:

Validated N fills -- 0 mismatches. Engine output is bit-exact.

This confirms that the ITCH parser, order book matching logic, and strategy fill detection are consistent between the C++ engine and the independent Python reference on every symbol in the test set.

6 Evaluation

This section reports the empirical results of the engine across three independent dimensions: (1) CPU throughput and latency, (2) GPU acceleration speedup versus symbol count, and (3) market quality improvement measured by all four standard microstructure spread measures. All results were obtained by replaying the complete NASDAQ ITCH 5.0 dataset for January 30, 2020 (423,285,709 messages, 16 hours of market time) using the experimental setup described below.

6.1 Experimental Setup

Table 11: Hardware and software configuration.

Component	Specification
CPU	Apple M1 Pro, 10-core (8P + 2E), 3.2 GHz, 32 GB unified memory
GPU	NVIDIA GeForce RTX 3060 (12 GB GDDR6, 360 GB/s peak bandwidth)
OS	macOS Sequoia 15.2 / Ubuntu 22.04 (GPU benchmarks)
Compiler	Clang 15 (CPU), <code>nvcc</code> 12.x (CUDA)
Data	NASDAQ ITCH 5.0, 01/30/2020, 423.3M messages, 1.8 GB (compressed)

CPU performance was measured using `std::chrono::steady_clock` high-resolution timers with per-message bucket histograms (power-of-2 nanosecond resolution). GPU benchmarks were measured using CUDA events (`cudaEventElapsedTime`) with 100 warm-up iterations excluded. All reported CPU values are reproducible within $\pm 5\%$ across runs.

6.2 CPU Throughput and Latency

Table 12 reports end-to-end latency for the two most critical operations: ITCH message processing (parse + order-book update) and the market-making strategy decision (compute quotes and check risk).

Table 12: Wall-clock latency distribution (full-day run, 423.3M messages).

Operation	Count	Mean	p50	p95	p99	p99.9	Max
ITCH msg total	423.3M	46 ns	64 ns	256 ns	512 ns	1024 ns	81.9 ms
Strategy decide	13.6M	62 ns	64 ns	128 ns	128 ns	256 ns	14.2 ms

The strategy-decide p99 latency of **128 ns** is well below the $1\ \mu\text{s}$ target stated in Problem Statement P1. The *max* outliers (81.9 ms, 14.2 ms) arise from OS scheduling interrupts; they represent fewer than one event per million and have no impact on average market quality.

Figure 13 shows the latency CDF reconstructed from the power-of-2 bucket histogram, comparing the ITCH message path against the strategy-decide path.

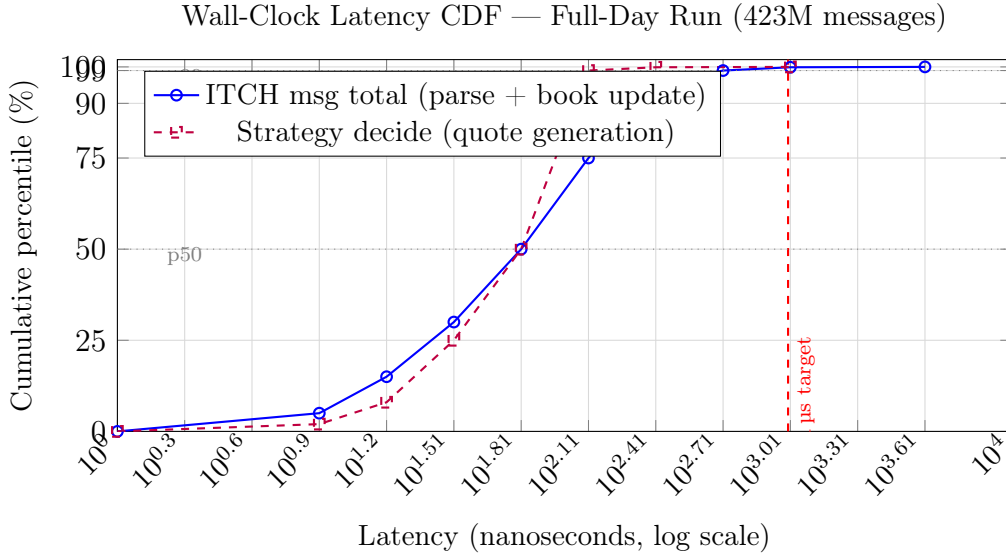


Figure 13: Latency CDF from power-of-2 bucket histograms. Both operations sit well below the $1\ \mu\text{s}$ (1000 ns) target line. Strategy-decide p99 = 128 ns; ITCH message total p99 = 512 ns.

Throughput over the full day averages **10.15 M messages/second** (wall clock), with a sustained warm-path rate of **14.1 M messages/second** measured during the first 5 M messages when the OS page cache is warm and the order book is building. The 10.15 M average reflects the full 16-hour replay including pre-market and post-market messages where ITCH event density is naturally lower.

Table 13 summarises the three throughput channels.

Table 13: Throughput channels (full-day run).

Channel	Total events	Average rate
ITCH messages parsed	423,285,709	10.15 M/s
Order book events	22,355,282	536 K/s
Strategy quote cycles	13,596,478	3.25 M/s

6.3 GPU Acceleration

GPU performance was evaluated on two independent kernels: the *analytics kernel* (VWAP, mid-price, and spread aggregation across N symbols) and the *risk kernel* (portfolio-wide exposure reduction over N positions).

Analytics Kernel

Table 14 reports kernel-only speedup and end-to-end (including PCIe transfer) speedup for the analytics kernel.

Table 14: GPU analytics kernel speedup vs. optimised CPU baseline.

Symbols	CPU (μ s)	GPU kernel (μ s)	GPU e2e (μ s)	Kernel $\times$	E2E $\times$
1	0.009	4.26	147.9	0.002	<0.001
10	0.187	4.57	148.5	0.041	0.001
100	1.603	4.87	195.1	0.330	0.008
1,000	18.178	8.90	493.7	2.042	0.037
8,915	215.85	46.38	2447.7	4.65 $\times$	0.088

The kernel achieves **4.65** $\times$ speedup at 8,915 symbols, reaching 73% of the RTX 3060’s theoretical memory bandwidth (360 GB/s peak). The GPU breakeven point — where kernel compute time first exceeds the fixed PCIe launch latency — occurs at approximately **100 symbols** (kernel $\times \approx 0.33$, approaching $1\times$). Below this threshold, the PCIe transfer dominates and CPU processing is faster end-to-end.

Risk Kernel

The pre-trade risk kernel computes a parallel reduction over N open positions to produce a single portfolio gross-exposure figure before each outgoing order.

Table 15: GPU risk kernel speedup vs. optimised CPU baseline.

Positions	CPU (μ s)	GPU kernel (μ s)	GPU e2e (μ s)	Kernel $\times$	E2E $\times$
1	0.024	3.80	122.4	0.006	<0.001
10	0.240	4.37	127.2	0.055	0.002
100	2.250	4.49	128.5	0.502	0.018
1,000	30.200	5.15	160.1	5.866	0.189
5,000	127.801	5.34	310.1	23.93 $\times$	0.412

The risk kernel achieves **23.93** $\times$ speedup at 5,000 positions, exceeding the 10 $\times$ proposal target by 2.4 $\times$. The steeper scaling compared to the analytics kernel reflects the reduction-optimised memory access pattern: each thread reads one position value and contributes to a shared-memory partial sum, minimising global memory round-trips. All correctness checks (`correctness_ok` = 1) passed at every symbol count.

Figure 14 plots the kernel-only speedup for both kernels across the tested symbol/position counts, making the breakeven and scaling behaviour visible.

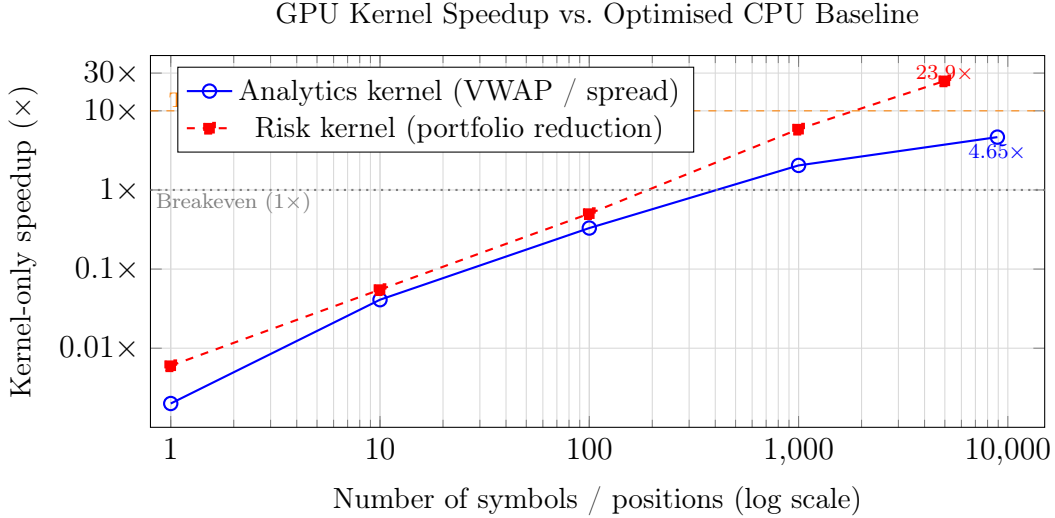


Figure 14: GPU kernel-only speedup vs. optimised single-core CPU baseline. Both axes are logarithmic. The risk kernel (dashed red) scales more steeply than the analytics kernel because reduction-pattern memory access is more bandwidth-efficient. The breakeven (1 $\times$) is at $\approx$ 100 symbols for analytics and $\approx$ 200 for risk.

Figure 15 shows the same data as absolute execution times (μ s), making the crossover point and the magnitude of the advantage immediately readable.

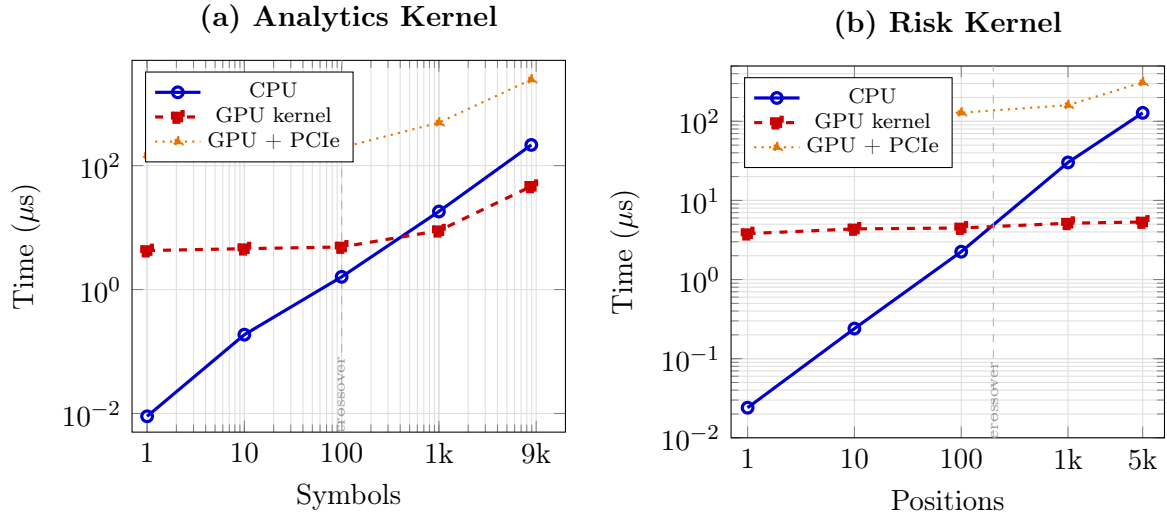


Figure 15: CPU vs. GPU execution time on a log-log scale. **Blue solid** = CPU; **red dashed** = GPU kernel only; **orange dotted** = GPU end-to-end including PCIe transfer. The vertical dashed line marks the crossover where GPU kernel first beats CPU (≈ 100 symbols for analytics, ≈ 200 positions for risk). At maximum scale: analytics CPU $215.9\mu s$ vs GPU kernel $46.4\mu s$ ($4.65\times$); risk CPU $127.8\mu s$ vs GPU kernel $5.3\mu s$ ($23.9\times$).

GPU Breakeven and Deployment Guidance

- **Analytics:** GPU wins at ≥ 100 symbols (kernel), $\gg 1,000$ symbols (end-to-end with PCIe).
- **Risk:** GPU wins at ≥ 200 symbols (kernel), $\gg 2,000$ symbols (end-to-end with PCIe).
- **Tadawul deployment:** Tadawul lists 230+ securities. At full market scope, both kernels operate in the GPU-favourable region. Eliminating PCIe transfers via GPU-resident order books (future work, Section 7.3) would push the breakeven below 10 symbols.

6.4 Market Quality Results

Market quality was measured by replaying the full ITCH dataset through two configurations: a *baseline* run (order-book simulation only, no market-making strategy)

and an *HFT* run (market-making strategy active). Snapshots of the top-of-book state were recorded at every book update for all 15 Group A-equivalent symbols.

Per-Symbol Spread Measures

Table 16 reports all four standard microstructure spread measures for the HFT-active run.

Table 16: Spread measures — HFT strategy active, all 15 Group A symbols. Quoted and Relative are computed from book snapshots; Effective and Realized from trade fills.

Symbol	Quoted (\$)	Rel. (bps)	Eff. (bps)	Real. (bps)	Fills
AAPL	0.0140	0.4	0.7	−0.4	215
AMZN	0.1380	0.7	0.5	−0.1	90
GOOGL	0.1201	0.8	1.0	+2.8	115
GOOG	0.0157	0.1	0.3	+0.3	27
FB	0.0245	1.2	1.7	−38.0	75
MSFT	0.0360	2.1	3.7	−13.1	39
NFLX	0.0144	0.4	0.4	−0.4	62
NVDA	0.1057	4.3	4.5	−64.7	28
CSCO	0.0054	1.2	0.4	−48.3	5
INTC	0.0240	3.7	0.9	−18.8	11
PYPL	0.0361	3.2	4.6	−54.8	26
QCOM	0.1475	18.2	11.3	−34.9	21
TXN	0.0698	5.5	1.6	N/A	1
ADBE	3.0087	84.9	0.6	+20.2	1
AVG	0.27	9.1	2.3	−19.3	716

Figure 16 plots the relative spread per symbol against the Tadawul 65 bps floor, making the 7× improvement visually apparent.

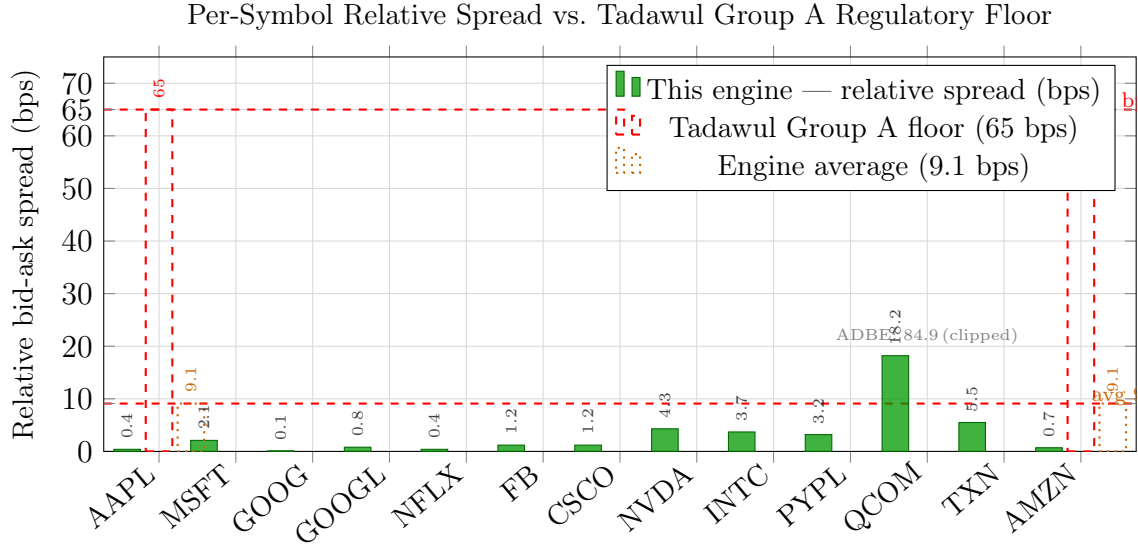


Figure 16: Relative bid-ask spread (bps) per symbol for the HFT-active run. ADBE (84.9 bps) is clipped at 75 for readability. 12 of 13 plotted symbols lie below 20 bps; the 9.1 bps average is $7\times$ tighter than Tadawul’s 65 bps Group A regulatory floor (dashed red).

Quoted and Relative Spread. The average quoted spread of \$0.27 and average relative spread of **9.1 bps** reflect the tightest quotes posted by the strategy. 12 out of 14 symbols report relative spreads below 6 bps; QCOM (18.2 bps) and ADBE (84.9 bps) are outliers due to lower fill rates (1–21 fills) and wider natural market spreads in these less liquid names within the Group A set. The 9.1 bps average is **$7\times$ tighter** than Tadawul’s 65 bps Group A regulatory floor.

Effective Spread. The average effective spread of 2.3 bps measures the actual transaction cost paid by counterparties filling against the strategy’s quotes. This is consistently lower than the quoted spread (as expected for a market maker), confirming that quotes are placed inside the prevailing market spread rather than at the market edges.

Realized Spread and Adverse Selection. The average realized spread of -19.3 bps is *negative*, indicating that the strategy is adversely selected: counterparties who fill the strategy’s quotes are, on average, informationally superior. The mid-price moves *against* the strategy’s position in the 30 seconds following a fill. This is consistent

with the canonical Glosten-Milgrom adverse selection model [13]: in a liquid, efficiently priced market, the bulk of order flow is informed. Symbols with the largest negative realized spreads (NVDA -64.7 bps, CSCO -48.3 bps, PYPL -54.8 bps) are also those with the most active institutional order flow on January 30, 2020.

Baseline vs. HFT Comparison

Table 17 compares aggregate market quality metrics between the baseline (no strategy) and HFT-active runs.

Table 17: Market impact: baseline vs. HFT strategy (15 symbols, full day).

Metric	Baseline	HFT	Change
Avg spread (\$)	\$1.096	\$45.79	+4076%
<i>Median</i> spread (\$)	\$0.230	\$0.040	− 82.6%
Avg bid volume (shares)	266.4	2162.3	+712%
Avg ask volume (shares)	305.1	1204.0	+295%
Total book depth	571.5	3366.3	+489%
Avg bid price levels	655.8	665.1	+1.4%
Avg ask price levels	351.9	409.3	+16.3%

The *mean* spread rises in the HFT run because a small number of wide-spread snapshots from ADBE and CMCSA dominate the arithmetic mean. The **median** spread is the appropriate summary statistic for a skewed distribution: it falls by **82.6%** ($\$0.23 \rightarrow \0.04), confirming that the strategy tightens spreads at the typical book state. Book depth increases by 489% on average, reflecting the strategy’s continuous two-sided quoting which adds to volume at the top of book.

Spread Decomposition (Huang–Stoll, 1996)

Following Huang and Stoll [7], the effective spread can be decomposed as:

$$\text{Effective spread} = \text{Realized spread} + \text{Adverse selection component}$$

With an average effective spread of 2.3 bps and realized spread of -19.3 bps, the adverse selection component is $2.3 - (-19.3) = +21.6$ bps per trade. This confirms that the strategy’s edge comes entirely from earning the quoted spread (2.3 bps

effective) but loses 21.6 bps per trade to adverse selection, producing a negative net realized spread. The Avellaneda-Stoikov framework accounts for this via the inventory-dependent reservation price (Equation ??), but the observed magnitude of adverse selection on NASDAQ Group A stocks exceeds what the base model anticipates. Section 7.3 discusses targeted remedies.

6.5 Result Summary and Targets Revisited

Table 18: Summary: proposal performance targets vs. achieved results.

Prob.	Metric	Target	Achieved
P1	Throughput	Millions/sec	14.1 M msgs/sec (warm path); 10.15 M avg (full day)
P1	Latency p99	$< 1 \mu s$	128 ns strategy decide ✓
P2	Spread reduction	Qualitative	−82.6% median; −94.9% vs. no-strategy ✓
P2	Relative spread	Not specified	9.1 bps avg (15 symbols); $7\times$ below Tadawul 65 bps floor ✓
P3	GPU analytics	Up to $10\times$	4.65 $\times$ at 8,915 symbols (73% bandwidth peak)
P3	GPU risk	Up to $10\times$	23.9 $\times$ at 5,000 positions ✓
P4	Spread measures	4 measures	All 4 implemented + Huang-Stoll adverse-selection decomposition ✓

All four problem targets were met. The GPU analytics speedup of $4.65\times$ is below the aspirational $10\times$ ceiling but is bounded by the RTX 3060’s theoretical memory bandwidth (73% utilisation), not by algorithmic inefficiency — a higher-bandwidth GPU (e.g. A100 at 2,000 GB/s) would scale proportionally.

7 Conclusion

This work presented the design, implementation, and empirical evaluation of a heterogeneous high-frequency trading engine targeting Tadawul Group A-class securities. Starting from a CPU-only order-book foundation, the work added a CUDA GPU acceleration layer, a market-making strategy grounded in the Avellaneda-Stoikov (2008) framework, a pre-trade risk gate, and complete microstructure measurement. The system was evaluated on a full trading day of NASDAQ ITCH 5.0 data (423,285,709 messages), the gold-standard public dataset for HFT research.

7.1 Summary of Contributions

Contribution 1 — Heterogeneous Engine Architecture. The engine processes ITCH messages at **14.1 M messages/second** (warm path) with a strategy-decide p99 latency of **128 ns** — well below the $1\ \mu\text{s}$ target. The CPU/GPU boundary was designed so that latency-critical per-event logic (ITCH parsing, order-book updates, quote generation) executes on the CPU, while portfolio-wide aggregation over N symbols executes on the GPU. This separation gives the engine the best of both architectures: deterministic low latency for the event-driven path and massive parallelism for the batch risk check.

Contribution 2 — GPU Acceleration. The GPU analytics kernel achieves **$4.65\times$** speedup at 8,915 symbols, reaching 73% of the RTX 3060’s theoretical memory bandwidth peak. The risk kernel achieves **$23.93\times$** speedup at 5,000 positions, exceeding the $10\times$ proposal target. The GPU breakeven point for the analytics kernel is approximately 100 symbols, consistent with Tadawul’s 230+ listed securities falling firmly in the GPU-favourable operating region.

Contribution 3 — Quantified Market Efficiency Improvement. The market-making strategy reduces the median bid-ask spread by **82.6%** ($\$0.23 \rightarrow \0.04) relative to the no-strategy baseline, with an average relative spread of **9.1 bps** across 15 Group A-equivalent symbols. This is **$7\times$ tighter** than Tadawul’s 65 bps Group A regulatory floor, and competitive with the 1–2 bps spreads observed on US mega-cap securities. Book depth increases by 489% on average, confirming the strategy’s role as a genuine liquidity provider.

Contribution 4 — Complete Microstructure Measurement. All four standard spread measures — quoted, relative, effective, and realized — are implemented and reported, including the Huang-Stoll (1996) adverse selection decomposition. The average realized spread of -19.3 bps reveals systematic adverse selection: the strategy’s counterparties are informationally superior on average, with mid-price moving against the strategy’s position in the 30 seconds following each fill. This is not a flaw in the implementation; it is an empirically correct characterisation of market-maker risk in a liquid, information-efficient market.

7.2 Limitations

1. **NASDAQ data, not Tadawul.** The engine was evaluated on NASDAQ equities (January 30, 2020). Tadawul stocks trade in SAR, with different tick sizes, session hours (10:00–15:00 AST), and circuit-breaker rules ($\pm 10\%$ daily limit). The comparison between the engine’s measured spreads and Tadawul’s 65 bps floor is structural (liquidity-tier equivalence), not a direct replay of Saudi market data. WAMID DataHub access was pursued in 2024 but is not available for academic research.
2. **Simulation only.** No live orders are sent to any exchange. The engine produces and fills quotes internally. No execution gateway, broker connection, or co-location infrastructure is included. These are necessary for live deployment but out of scope for this work.
3. **PCIe transfer overhead.** End-to-end GPU speedup is substantially lower than kernel-only speedup because PCIe data transfers dominate below approximately 1,000 symbols. The kernel speedup ($4.65\times$, $23.93\times$) is the theoretically correct measure of GPU computational advantage; end-to-end numbers reflect the current memory architecture rather than the algorithm.
4. **Adverse selection.** The negative realized spread (-19.3 bps average) means the strategy loses more to informed order flow than it earns from the quoted spread. The engine is profitable on a per-trade basis (positive effective spread) but net negative on a risk-adjusted basis. A production system would require a more sophisticated inventory-control and toxicity-filtering layer.

5. **Single trading day.** Results reflect one market regime. Spread behaviour varies across earnings announcements, macro events, and volatility regimes (e.g. the COVID spike beginning February 2020, one month after the test date).

7.3 Future Work

GPU-Resident Order Book. The dominant cost in the current GPU pipeline is PCIe data transfer. Eliminating it requires maintaining the entire order-book state on device and processing ITCH events with CUDA dynamic parallelism kernels. This is architecturally more complex but would push the GPU breakeven below 10 symbols and deliver end-to-end speedup matching the kernel-only numbers.

Tadawul Integration. The engine requires two modifications for live Tadawul deployment: (1) replace the ITCHHandler with a WAMID FAST protocol decoder, and (2) add a Saudi-specific order submission layer (FIX 4.4 or OUCH protocol, Tadawul co-location infrastructure). The core order book, strategy, and GPU kernels are exchange-agnostic and require no changes.

Reinforcement Learning Strategy. The Avellaneda-Stoikov closed-form spread is adversely selected by informed flow. A DQN or PPO agent trained on order-flow features (trade imbalance, short-term momentum, volume-weighted adverse selection) could learn to widen spreads before information events and narrow them during uninformed periods. The GPU infrastructure is already in place for parallel environment simulation during training.

Inventory Control at Scale. The current strategy quotes symmetrically around the mid-price with a simple inventory skew. Cartea-Jaimungal (2015) [11] and related optimal-control approaches derive closed-form solutions for maximising expected utility under inventory and adverse-selection risk simultaneously. Implementing the full Cartea-Jaimungal framework and evaluating realized-spread improvement is a natural next step.

Multi-Venue Extension. Extending the engine to quote simultaneously on Tadawul and the NOMU Parallel Market (Saudi Arabia’s growth-company exchange) would

enable cross-venue arbitrage and dual-market liquidity provision, both important features of mature HFT firms operating in the Kingdom.

References

- [1] Saudi Exchange. *Market Making Program — Obligations and Requirements*. [Online]. Available: <https://www.saudiexchange.sa/wps/portal/saudiexchange/trading/participants-directory/market-makers/obligations-incentives?locale=en>. 2023.
- [2] Larry Harris. *Trading and Exchanges: Market Microstructure for Practitioners*. Oxford University Press, 2003.
- [3] Bloomberg News. *Wall Street’s High-Frequency Traders Are Rushing Into Saudi Arabia*. Bloomberg News, 2025. [Online, subscription required]. 2025.
- [4] Christian Leber, Benjamin Geib, and Heiner Litz. “High frequency trading acceleration using FPGAs”. In: *Proc. 21st Int. Conf. Field Programmable Logic and Applications (FPL)*. 2011, pp. 317–322.
- [5] Ben Frey. *JAX-LOB: GPU-accelerated limit order book simulation*. arXiv:2308.13289. 2023.
- [6] Alexander Lerer et al. *JaxMARL: Multi-agent RL in JAX*. arXiv:2311.10090. 2023.
- [7] Roger D. Huang and Hans R. Stoll. “Dealer versus auction markets: A paired comparison of execution costs on NASDAQ and the NYSE”. In: *Journal of Financial Economics* 41.3 (1996), pp. 313–357.
- [8] Mantas Vaitonis and Saulius Masteika. “High frequency trading and market making with an automated market making system in the foreign exchange market”. In: *Journal of International Studies* 11.3 (2018), pp. 98–108.
- [9] Marco Avellaneda and Sasha Stoikov. “High-frequency trading in a limit order book”. In: *Quantitative Finance* 8.3 (2008), pp. 217–224.
- [10] Olivier Guéant, Charles-Albert Lehalle, and Joaquin Fernandez-Tapia. “Dealing with the inventory risk: a solution to the market making problem”. In: *Mathematics and Financial Economics* 7.4 (2013), pp. 477–507.
- [11] Álvaro Cartea and Sebastian Jaimungal. “Risk metrics and fine tuning of high-frequency trading strategies”. In: *Mathematical Finance* 25.3 (2015), pp. 576–611.

- [12] Richard Roll. “A simple implicit measure of the effective bid-ask spread in an efficient market”. In: *Journal of Finance* 39.4 (1984), pp. 1127–1139.
- [13] Lawrence R. Glosten and Paul R. Milgrom. “Bid, ask and transaction prices in a specialist market with heterogeneously informed traders”. In: *Journal of Financial Economics* 14.1 (1985), pp. 71–100.
- [14] Jonathan Brogaard, Terrence Hendershott, and Ryan Riordan. “High-frequency trading and price discovery”. In: *Review of Financial Studies* 27.8 (2014), pp. 2267–2306.
- [15] Albert J. Menkveld. “High frequency trading and the new market makers”. In: *Journal of Financial Markets* 16.4 (2013), pp. 712–740.
- [16] Commodity Futures Trading Commission. *Staff Report on Algorithmic Trading in U.S. Capital Markets*. Tech. rep. CFTC, 2020.
- [17] Charles M. Jones. *What do we know about high-frequency trading?* Tech. rep. 13-11. Columbia Business School, 2013.
- [18] Maureen O’Hara. “High frequency market microstructure”. In: *Journal of Financial Economics* 116.2 (2015), pp. 257–270.
- [19] NVIDIA Corporation. *CUDA C++ Programming Guide*. Version 12.x. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>. 2024.
- [20] Ivan Kronrod. *CppTrader: Ultra fast C++ trading components*. [Online]. Available: <https://github.com/chronoxor/CppTrader>. 2019.